

DATATALKSCLUB

MLZOOMCAMP 2025

Week 5 notes

Carlos Leal León

October 27, 2025

General Index

1	Deployment Overview	3
2	Train, Save and Load the Model	3
3	Flask	4
3.1	Churn model to Flask Web Service	4
4	Pipenv	7
5	FastAPI and uv	7
6	Homework 5 Notes	8
6.1	Installing uv	8
6.2	Initialize uv project	8
6.3	Install package in uv	8
6.3.1	Scikit-Learn hash	8
6.4	Loading pipeline with Pickle	8
6.5	Model as Web Service using FastAPI	9
7	Docker	11
7.1	Docker file	11
7.2	Dockerfile Build	13
7.3	Run built docker image	13
7.4	Docker pull	16
7.5	Inspecting docker image	16
7.6	Building downloaded docker image	18

Figure Index

1	Docker status.	11
2	Docker build.	13
3	Docker run.	14
4	Service running request.	14
5	Service running response.	15
6	Docker pull.	16
7	Docker images.	16
8	Docker image filesystem.	17
9	Downloaded image build response output.	19
10	Downloaded image run output.	19
11	Downloaded image build request.	20
12	Service request and response http codes.	20
13	Downloaded image build response.	21

1 Deployment Overview

So far you have used jupyter notebook to use a churn prediction model. If you want to use the model and not have to run again all the code to make a new prediction you have to deploy the model.

Model deployment involves using a server to deploy the code of the model and through endpoints connect to the server to make predictions using new data.

The goal of all of this comes with an advantage, the chance of automating certain actions or processes based on the prediction results.

2 Train, Save and Load the Model

First of all you have to train a model. From previous weeks you have trained a model for churn prediction. In the process you dealt with Exploring the dataset, Data Preparation, Feature Engineering, Feature Encoding, Standardization, Data Partitioning, Model Training, Validation and Test. Some steps were easier than others but in the end the result is the best available trained model.

Once the final model is trained you have to save that model. As the model is the best model at this point you want to be able to use the available model without going through the whole training process every time a new prediction is required.

In order to save the model you do:

- pickle: First install pickle-mixin using *pip install pickle-mixin*
- save: Write the model as bytes to a file

Listing 1: Write model to a file.

```
1 import pickle
2
3 #save model as binary to a file
4 with open("model_name.bin", "wb") as f_out:
5     pickle.dump((dv, model), f_out)
```

NOTE: It is important to highlight that the encoding process is done using the DictVectorizer referenced by dv so in case more pre-processing is done or required the tuple has to contain the necessary elements that the model uses to get data ready to make predictions.

- load: Read model from saved file

Listing 2: Read model from file.

```
1 import pickle
2
3 #load model from file
4 with open("model_name.bin", "rb") as f_in:
5     dv, model = pickle.load(f_in)
```

3 Flask

Here you are introduced to a simple web service with an example of a function returning "PONG" using flask [1]. You have to install flask in order to turn the function into a simple web service. To install flask do *pip install flask*.

The code is similar to:

Listing 3: Pong function to simple web service.

```
1  #!/usr/bin/env python3
2  from flask import Flask
3
4  ping_ws = Flask("ping_ws")
5
6  @ping_ws.route("/ping_ws", methods=["GET"])
7  def ping():
8      return "PONG"
9
10 def main():
11     ping_ws.run(debug=True, host="0.0.0.0", port=4444)
12
13 if __name__ == "__main__":
14     main()
```

I added a shebang at the top to make the main.py file executable using *./* and do not forget to add execution permissions.

Later you initialize your web service I used ping_ws. Remember that *ws.run* must run in the main scope.

To execute it I did *./main.py* from the location of the *main.py* file from a terminal.

And to test it I used *0.0.0.0:4444/ping_ws* from the browser.

3.1 Churn model to Flask Web Service

From the previous examples now its turn to create a web service for the churn prediction model here you have to combine code shown previously and modify it to fit the prediction churn model needs.

The steps are simple:

- Load previously saved model
- Define a function to make a prediction on a new customer using the model
- Define a function that is going to be used to create the web service which will call the model to make predictions
- Send a post request using customer data in json format remember you are using POST and review the response.

The code looks similar to:

Listing 4: Load model from file.

```
1 import pickle
2
3 with open("your_churn_model.bin", "rb") as f_in:
4     dv, model = pickle.load(f_in)
```

Listing 5: Funtion to make new predictions.

```
1 def predict_single(customer, dv, model):
2     #remember to encode categorical features of new observation
3     X = dv.transform([customer])
4     y_pred = model.predict_proba(X)[:, 1]
5     return y_pred[0]
```

Listing 6: Function to create web service.

```
1 predict_churn_ws = Flask("predict_churn_ws")
2
3 @predict_churn_ws.route("/predict_churn_ws", methods=["POST"])
4 def predict():
5     customer = request.get_json()
6
7     prediction = predict_single(customer, dv, model)
8     churn = prediction >= 0.5
9
10    result = {
11        #casts here are important, described in the lecture videos
12        "churn_probability": float(prediction),
13        "churn": bool(churn)
14    }
15
16    return jsonify(result)
```

Listing 7: POST customer data as json.

```

1 import requests
2
3 #url of predict_churn_ws
4 url = "http://localhost:4444/predict_churn_ws"
5
6 #simulate new customer data
7 customer = {
8     "customerid": "8879-zkjof",
9     "gender": "female",
10    "seniorcitizen": 0,
11    "partner": "no",
12    "dependents": "no",
13    "tenure": 41,
14    "phoneservice": "yes",
15    "multiplelines": "no",
16    "internetservice": "dsl",
17    "onlinesecurity": "yes",
18    "onlinebackup": "no",
19    "deviceprotection": "yes",
20    "techsupport": "yes",
21    "streamingtv": "yes",
22    "streamingmovies": "yes",
23    "contract": "one_year",
24    "paperlessbilling": "yes",
25    "paymentmethod": "bank_transfer_(automatic)",
26    "monthlycharges": 79.85,
27    "totalcharges": 3320.75
28 }
29
30 #make a request
31 response = requests.post(url, json=customer)
32 result = response.json()
33 print(result)

```

Up to this point this is the development version of the churn web service.

To run it as a production ready web service in the lectures one of the available options to use is gunicorn [2].

To use it you do:

- Install: *pip install gunicorn* This is for linux kernel based os.
- Run service: *gunicorn --bind <ip>:<port> name:app*
- Where:
 - name: Is the name used in *app = Flask("name")*
 - It may be required to adjust it as you named it in your file.

4 Pipenv

One way to handle projects with different dependencies is to isolate them from each other using virtual environments that way system dependencies and dependencies for different projects do not affect each other. There are several tools to achieve this but one recommended option is *pipenv* [3].

- Install: Do *pip install pipenv*
- Use packages: *pipenv install < packagename >*

When you install a package it is added to two files:

- Pipfile: Manages dependencies with their specific versions.
- Pipfile.lock: Includes a hash for the dependency specified to use the exact version of each dependency.

Now to run code from your virtual environment:

- shell: Do *pipenv shell* from your project location
- run a command in shell i.e.: Do *pipenv run gunicorn --bind < ip >:< port > name:app*

5 FastAPI and uv

This is an updated version of a similar web service deployment using newer tools [4]. The reference is a workshop where it explains the whole process of serving a churn model. I am not breaking it down in my notes it is better to consult it and go from there so you can have your own experience with the workshop. The workshop is used to later answer questions from homework 5, so if you want to take a look go to the next section.

6 Homework 5 Notes

This homework requires to use uv [5], FastAPI [6], code from previous subjects and homework as many of the steps require you to know what you are supposed to do and all of that has been covered previously. I am breaking down some of the steps so I can easily remember what I did.

6.1 Installing uv

uv is replacing pipenv. Install uv system-wide then from an empty folder execute the upcoming commands. This will save you headaches.

- Install: *pip install uv*
- Version: *uv --version*

6.2 Initialize uv project

An initialized empty uv project is recommended, execute the next command from an empty folder of your choice.

- init: *uv init project-name*

Project name is not strictly required as long as you use it inside of the folder you want to initialize is fine.

6.3 Install package in uv

To add a package using uv you can add it using the package name or using the package name and a specific version.

- any: *uv add <package-name>==<version>*
- example: *uv add scikit-learn==1.6.1*

6.3.1 Scikit-Learn hash

After you install one package the uv.lock file is generated, in case you want to trigger it manually use:

- lockfile: *uv lock*

Then you can use cat or less to see its content.

6.4 Loading pipeline with Pickle

To load the pipeline.bin file remember that you have to train your model first as in the previous homework and once you selected the final model you save that model as a pipeline instead of the tuple (dv, model) as described in the workshop. In the instructions you are provided with a sample pipeline so you can load it in case you do not have a saved pipeline of your own. The loaded pipeline is then used to predict the probability of a customer churning.

6.5 Model as Web Service using FastAPI

In the workshop you use FastAPI and uvicorn [7].

- Install: `uv add fastapi`
- Install: `uv add uvicorn`

Once installed write code to serve the model. As this homework provides the pipeline I only included simple code used to predict.

Listing 8: Serve model using FastAPI.

```

1 import pickle
2 import uvicorn
3
4 from fastapi import FastAPI
5 from typing import Dict, Any
6
7 #load model
8 with open("pipeline_v1.bin", "rb") as pipe_in:
9     pipeline = pickle.load(pipe_in)
10
11 #predict single observation
12 def predict_single(observation):
13     score = pipeline.predict_proba(observation)[0, 1]
14     return float(score)
15
16 predict_churn_telco = FastAPI(title="predict_churn_telco")
17
18 @predict_churn_telco.post("/predict_churn_telco")
19 def predict(observation: Dict[str, Any]):
20     score = predict_single(observation)
21     return {"score": score, "is_churn": bool(score >= 0.5)}
22
23 def main():
24     #uvicorn.run("main:predict_churn_telco", host="127.0.0.1", port=4444)
25     uvicorn.run("main:predict_churn_telco", host="0.0.0.0", port=4444)
26
27 if __name__ == "__main__":
28     main()

```

Now you have to be able to send a request to the deployed model. You can use curl or requests. In case you want to use requests install it from your uv project folder.

Listing 9: Use requests to call served model.

```
1 import requests
2
3 url = ("http://127.0.0.1:4444/predict_churn_telco")
4
5 observation = {
6     "lead_source": "organic_search",
7     "number_of_courses_viewed": 4,
8     "annual_income": 80304.0
9 }
10
11 response = requests.post(url, json=observation)
12 result = response.json()
13
14 print(result)
15 print("About_to_Churn" if result["is_churn"] else "No_Churn")
```

Now to test it.

- Run server: *uv run main.py*
- Run test: *uv run client.py*

Run each command from the corresponding file location.

You should get something like:

'score': 0.5340417283801275, 'is_churn': True

About to Churn

7 Docker

Once the service has been deployed using uv and FastAPI is time to pack it in a container. To do that you have to install Docker [8], the installation process may vary depending on your os so you need to look for specific instructions.

I followed the instructions to install from docker repository to get an updated stable version, you can install it from your os repositories just check if the version is actively maintained and updated otherwise you may have to install it as I did.

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2025-10-24 18:28:12 CST; 8s ago
 TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 16934 (dockerd)
       Tasks: 13
      Memory: 22.7M
         CPU: 279ms
    CGroup: /system.slice/docker.service
            └─16934 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Figure 1: Docker status.

7.1 Docker file

The docker file contains the instructions to 'automatically' build the container image in this case of the served model. Here you specify the base image, dependencies, files to copy, set environment variables.

My docker file looks like this:

Listing 10: Docker file.

```
1 FROM python:3.10.12-slim-bookworm
2
3 COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
4 WORKDIR /code
5
6 ENV PATH="/code/.venv/bin:$PATH"
7
8 COPY "pyproject.toml" "uv.lock" ".python-version" ./
9 RUN uv sync --locked
10
11 COPY "main.py" "pipeline_v1.bin" ./
12
13 EXPOSE 4444
14
15 ENTRYPOINT ["uvicorn", "main:predict_churn_telco", "--host", "0.0.0.0", "--port", "4444"]
```

This Dockerfile is located in the root of your uv virtual environment.

Where:

- **FROM:** In a Dockerfile specifies the base image that serves as the foundation for building the new Docker image.
- **COPY:** In a Dockerfile is used to copy files or directories from the build context (the local directory containing the Dockerfile) into the filesystem of the Docker image being built.
- **ENV:** The ENV instruction in a Dockerfile sets an environment variable within the Docker image during the build process. These variables are available to any subsequent RUN, CMD, ENTRYPOINT, or other instructions that follow in the Dockerfile. The primary purpose of ENV is to define default values for environment variables that will be accessible both during the build phase and at runtime when a container is launched from the resulting image.
- **RUN:** The RUN instruction in a Dockerfile is used to execute commands during the image build process. It creates a new layer on top of the current image by running the specified command and committing the results to the new layer. This instruction is typically used to install software packages, update system configurations, create directories, or perform other setup tasks required for the application.
- **EXPORT:** The EXPORT instruction in a Dockerfile declares which network ports a container will listen on at runtime, serving as a form of documentation for the intended network usage of the application inside the container. It does not actually publish or open the port on the host machine; instead, it informs users and tools about the ports that should be exposed or published when the container is run.
- **ENTRYPOINT:** The ENTRYPOINT instruction in a Dockerfile specifies the primary command that will always run when a container is started from the resulting image, defining the container's core function. It ensures that a specific executable, such as a web server, database, or application script, is executed by default upon container initialization.

7.2 Dockerfile Build

Once the Dockerfile is ready is time to build it. To do that you use: *docker build -t <name:tag>* .

- *docker build*: Used to create an image from a Dockerfile.
- *-t*: Specifies a tag for the image, if no tag is provided defaults to *'latest'*.
- *.* : The dot at the end means from the current directory.

My command is: *docker build -t predict_churn_telco* .

If you do not add your os user to the docker group you have to use *sudo* to execute each docker command. The output is the following:

```
[*] Building 28.6s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 394B
=> [internal] load metadata for ghcr.io/astral-sh/uv:latest
=> [internal] load metadata for docker.io/library/python:3.10.12-slim-bookworm
=> [internal] load .dockerignore
=> => transferring context: 2B
=> FROM ghcr.io/astral-sh/uv:latest@sha256:f459f6f73a8c4ef5d69f4e6fbbdb8af751d6fa40ec34b39a1ab469acd6e289b7
=> => resolve ghcr.io/astral-sh/uv:latest@sha256:f459f6f73a8c4ef5d69f4e6fbbdb8af751d6fa40ec34b39a1ab469acd6e289b7
=> sha256:99df4935c9790c120a2cd9be361f1d2717560ba4c68e6eb037a883bfac214c18 96B / 96B
=> sha256:f459f6f73a8c4ef5d69f4e6fbbdb8af751d6fa40ec34b39a1ab469acd6e289b7 2.19kB / 2.19kB
=> sha256:0f419824ea1810fe2a47af12aef8e8c39ea8e8d19c728cd9612d18e17a98d17 669B / 669B
=> sha256:d113707eb731275ee0cdffb9c115f020876888fa376857bee3f6cc5a471e2f2b 1.30kB / 1.30kB
=> sha256:0daf2b1a974db1a0f3bca91305aba6c54d33d82104cd404c9c58813b6dd4146 21.82MB / 21.82MB
=> => extracting sha256:0daf2b1a974db1a0f3bca91305aba6c54d33d82104cd404c9c58813b6dd4146
=> => extracting sha256:99df4935c9790c120a2cd9be361f1d2717560ba4c68e6eb037a883bfac214c18
=> [internal] load build context
=> => transferring context: 164.90kB
=> [stage-0 1/6] FROM docker.io/library/python:3.10.12-slim-bookworm@sha256:4d440b214e447deddc0a94de23a3d97d28dfafdf125a8b4bb8073381510c9ee2
=> => resolve docker.io/library/python:3.10.12-slim-bookworm@sha256:4d440b214e447deddc0a94de23a3d97d28dfafdf125a8b4bb8073381510c9ee2
=> sha256:4d440b214e447deddc0a94de23a3d97d28dfafdf125a8b4bb8073381510c9ee2 1.65kB / 1.65kB
=> sha256:13cc670c11ee0d0ba92d95f35f4d8e991489371f03ba04788e93268bf9e9d2e 1.37kB / 1.37kB
=> sha256:f31284a067273a64cc5b0e64e2a613de45d817d9094b02d37d6bf522933b16 6.92kB / 6.92kB
=> sha256:52d2b7f179e32b4c0d579ee3c4958027988f9a8274850ab0c7c24661e3adaac5 29.12MB / 29.12MB
=> sha256:2b8a9a2240c1224b34f6aafbc310f9a3fe65bd6893050906d02e89fc8326aa9 3.50MB / 3.50MB
=> => extracting sha256:52d2b7f179e32b4c0d579ee3c4958027988f9a8274850ab0c7c24661e3adaac5
=> sha256:618a49bbc6c68a58b082ceb072a9464370d0203a55c70e9cfc16caf6c3c8f383 17.43MB / 17.43MB
=> => extracting sha256:618a49bbc6c68a58b082ceb072a9464370d0203a55c70e9cfc16caf6c3c8f383
=> sha256:708d13076071a7b9fb3eebf4a360636911b7d83b3bb1939165eb3d0136f4aa84 243B / 243B
=> sha256:90b76fc2ebde9ad1526763819788ed8ea1817179c0520cd9e5e9362bc6b35155 3.37MB / 3.37MB
=> => extracting sha256:2b8a9a2240c1224b34f6aafbc310f9a3fe65bd6893050906d02e89fc8326aa9
=> => extracting sha256:618a49bbc6c68a58b082ceb072a9464370d0203a55c70e9cfc16caf6c3c8f383
=> => extracting sha256:708d13076071a7b9fb3eebf4a360636911b7d83b3bb1939165eb3d0136f4aa84
=> => extracting sha256:90b76fc2ebde9ad1526763819788ed8ea1817179c0520cd9e5e9362bc6b35155
=> [stage-0 2/6] COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
=> [stage-0 3/6] WORKDIR /code
=> [stage-0 4/6] COPY pyproject.toml uv.lock .python-version ./
=> [stage-0 5/6] RUN uv sync --locked
=> [stage-0 5/6] COPY main.py pipeline_v1.bin ./
=> => exporting image
=> => exporting layers
=> => writing image sha256:f306af03bcd6ee031145f0a2276b312474524bf07f60a4e19e2e78ebf52983e0
=> => naming to docker.io/library/predict_churn_telco
```

Figure 2: Docker build.

7.3 Run built docker image

Once built you run it using: *docker run -it --rm -p <host_port>:image_port <name:tag>*

- *docker run*: Start a docker container.
- *-it*: Allocates a pseudo-TTY and keep stdin open for an interactive session.
- *--rm*: Instructs Docker to automatically remove the container and its file system after it exits.
- *-p*: Maps a port on the host machine (Docker host) to a port inside the container (Docker image).
- *name:tag* : Is the name used to build the Dockerfile.

Note: The port of the host must be available as well as the port on the Docker image.

My command is: *docker run -it --rm -p 4444:4444 predict_churn_telco*.

The output is the following:

```
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:4444 (Press CTRL+C to quit)
```

Figure 3: Docker run.

Note: Be aware that using localhost address yields connection reset, I had to go back and change it to listen on all addresses. Though your file containing the FastAPI service can have the address set to localhost but not the Dockerfile, I still have to figure this out as what you want to do is expose the container to the outside but not on all addresses right?

If you access to the url you can see:

predict_churn_telco 0.1.0 OAS 3.1

/openapi.json

default

POST /predict_churn_telco Predict

Parameters

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "lead_source": "organic_search",
  "number_of_courses_viewed": 4,
  "annual_income": 80304.0
}
```

Execute Clear

Figure 4: Service running request.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:4444/predict_churn_telco' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "lead_source": "organic_search",
    "number_of_courses_viewed": 4,
    "annual_income": 80304.0
  }'
```

Request URL

http://127.0.0.1:4444/predict_churn_telco

Server response

Code	Details
200	Response body <pre>{ "score": 0.5340417283801275, "is_churn": true }</pre> Response headers <pre>content-length: 44 content-type: application/json date: Sat, 25 Oct 2025 03:54:48 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Figure 5: Service running response.

7.4 Docker pull

Docker pull command is used to download images or repositories from a registry to the local machine. By default Docker pulls from Docker Hub unless a different registry is specified. The command retrieves images based on their name and optional tag.

Example: `docker pull agrigorev/zoomcamp-model:2025`

I used sudo to execute the command and the output looks like:

```
2025: Pulling from agrigorev/zoomcamp-model
59e22667830b: Pull complete
19f81082f6f0: Pull complete
799df5177ae2: Pull complete
66cd357cf7a1: Pull complete
760c2b9cc6ee: Pull complete
816e9c0578b5: Pull complete
Digest: sha256:14d79fde0bbf078eb18c99c2bd007205917b758ec11060b2994963a1e485c2ae
Status: Downloaded newer image for agrigorev/zoomcamp-model:2025
docker.io/agrigorev/zoomcamp-model:2025
```

Figure 6: Docker pull.

Now you can take a look at the images:

```
es$ sudo docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
predict_churn_telco latest      40bbe567224a 37 hours ago 432MB
agrigorev/zoomcamp-model 2025       4a9ecc576ae9 5 days ago   121MB
```

Figure 7: Docker images.

Note: If you have rebuilt the same image you may have entries looking like `<none:none>` to remove them you can use `docker image prune`.

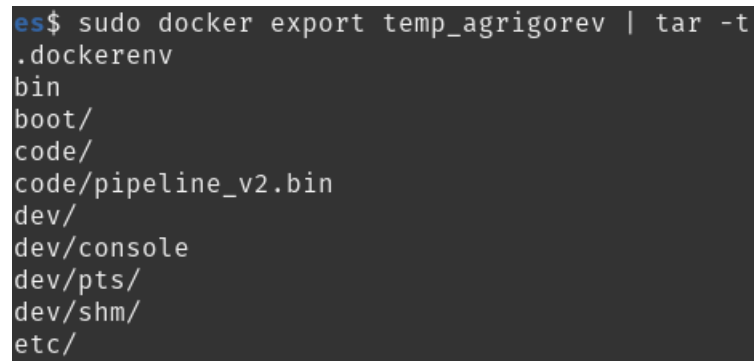
7.5 Inspecting docker image

You are asked to build the downloaded Docker image again and it is required to know what is the name of the file that contains the model as the FastAPI script used it to predict.

What I did:

- Temporary container: `docker create --name <temp_name> <image_id>`
- Export filesystem:
 - To file: `docker export <temp_name> | tar -t > <file_name>.txt`
 - To stdout: `docker export <temp_name> | tar -t`

Then you can inspect either file or terminal to look at the content of the downloaded image.

A terminal window with a dark background. The command 'es\$ sudo docker export temp_agrigorev | tar -t' has been executed. The output lists the contents of the Docker image's filesystem: .dockerenv, bin, boot/, code/, code/pipeline_v2.bin, dev/, dev/console, dev/pts/, dev/shm/, and etc/.

```
es$ sudo docker export temp_agrigorev | tar -t
.dockerenv
bin
boot/
code/
code/pipeline_v2.bin
dev/
dev/console
dev/pts/
dev/shm/
etc/
```

Figure 8: Docker image filesystem.

And once reviewed you can remove the temporary container:

- Remove temporary container: `docker rm <temp_name>`

7.6 Building downloaded docker image

- Modify main line 8:

Listing 11: main file edit.

```

1 import pickle
2 import uvicorn
3
4 from fastapi import FastAPI
5 from typing import Dict, Any
6
7 #load model, edited this line
8 with open("pipeline_v2.bin", "rb") as pipe_in:
9     pipeline = pickle.load(pipe_in)
10
11 #predict single observation
12 def predict_single(observation):
13     score = pipeline.predict_proba(observation)[0, 1]
14     return float(score)
15
16 predict_churn_telco = FastAPI(title="predict_churn_telco")
17
18 @predict_churn_telco.post("/predict_churn_telco")
19 def predict(observation: Dict[str, Any]):
20     score = predict_single(observation)
21     return {"score": score, "is_churn": bool(score >= 0.5)}
22
23 def main():
24     uvicorn.run("main:predict_churn_telco", host="0.0.0.0", port
25               =4444)
26
27 if __name__ == "__main__":
28     main()

```

- Modify Dockerfile lines 1 and 11:

Listing 12: Dockerfile edit.

```

1 FROM agrigorev/zoomcamp-model:2025
2
3 COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
4 WORKDIR /code
5
6 ENV PATH="/code/.venv/bin:$PATH"
7
8 COPY "pyproject.toml" "uv.lock" ".python-version" ./
9 RUN uv sync --locked
10
11 COPY "main.py" ./
12
13 EXPOSE 4444
14
15 ENTRYPOINT ["uvicorn", "main:predict_churn_telco", "--host", "0.0.0.0",
16           ", --port", "4444"]

```

- Build image: `sudo docker build -t agrigorev/zoomcamp-model .`

```
wide/hw_05$ sudo docker build -t agrigorev/zoomcamp-model .
[+] Building 15.5s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 375B                                0.0s
=> [internal] load metadata for ghcr.io/astral-sh/uv:latest        0.7s
=> [internal] load metadata for docker.io/agrigorev/zoomcamp-model:2025 0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> FROM ghcr.io/astral-sh/uv:latest@sha256:f459f6f73a8c4ef5d69f4e6fbbdb8af751d6fa40ec3 0.0s
=> [stage-0 1/6] FROM docker.io/agrigorev/zoomcamp-model:2025    0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 956B                                      0.0s
=> CACHED [stage-0 2/6] COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/ 0.0s
=> CACHED [stage-0 3/6] WORKDIR /code                              0.0s
=> [stage-0 4/6] COPY pyproject.toml uv.lock .python-version ./ 0.0s
=> [stage-0 5/6] RUN uv sync --locked                             12.7s
=> [stage-0 6/6] COPY main.py ./                                   0.0s
=> exporting to image                                              2.0s
=> => exporting layers                                              2.0s
=> => writing image sha256:ce4b7f95138f8e663405b08c463fd36e8936e8ef64b27c1dcfe993f80bb 0.0s
=> => naming to docker.io/agrigorev/zoomcamp-model                0.0s
```

Figure 9: Downloaded image build response output.

- Run image: `sudo docker run -it --rm -p 4444:4444 agrigorev/zoomcamp-model`

```
wide/hw_05$ sudo docker run -it --rm -p 4444:4444 agrigorev/zoomcamp-model
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:4444 (Press CTRL+C to quit)
```

Figure 10: Downloaded image run output.

Once launched you can review the request:

The screenshot shows a REST client interface with a title bar 'default'. The main area displays a POST request to the endpoint '/predict_churn_telco' with the action 'Predict'. Below the endpoint, there is a 'Parameters' section with 'No parameters' and buttons for 'Cancel' and 'Reset'. The 'Request body' section is marked as 'required' and has a dropdown menu set to 'application/json'. The request body content is a JSON object:

```
{  "lead_source": "organic_search",  "number_of_courses_viewed": 4,  "annual_income": 80304.0}
```

. At the bottom, there are 'Execute' and 'Clear' buttons.

Figure 11: Downloaded image build request.

Here you can take a look at the service handling the request.

```
INFO:      Uvicorn running on http://0.0.0.0:4444 (Press CTRL+C to quit)
INFO:      172.17.0.1:38184 - "GET /docs HTTP/1.1" 200 OK
INFO:      172.17.0.1:38184 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      172.17.0.1:35316 - "POST /predict_churn_telco HTTP/1.1" 200 OK
```

Figure 12: Service request and response http codes.

And review the response:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:4444/predict_churn_telco' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "lead_source": "organic_search",
    "number_of_courses_viewed": 4,
    "annual_income": 80304.0
  }'
```

Request URL

```
http://127.0.0.1:4444/predict_churn_telco
```

Server response

Code	Details
200	Response body <pre>{ "score": 0.9933071490756734, "is_churn": true }</pre> Response headers <pre>content-length: 44 content-type: application/json date: Sun, 26 Oct 2025 18:32:28 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Figure 13: Downloaded image build response.

As you can see the sample data is the same as in Figure 4 but in this case the result is different as the model used is different.

References

- [1] Flask. "Flask" [Online], at: flask [Consulted on October 2025].
- [2] gunicorn. "gunicorn" [Online], at: gunicorn [Consulted on October 2025].
- [3] Pipenv. "Pipenv" [Online], at: pipenv [Consulted on October 2025].
- [4] DataTalksClub (github). "05-deployment Workshop" [Online], at: 05-deployment-workshop [Consulted on October 2025].
- [5] astral-sh (github). "uv" [Online], at: uv [Consulted on October 2025].
- [6] FastAPI. "FastAPI" [Online], at: fastapi [Consulted on October 2025].
- [7] Uvicorn. "uvicorn" [Online], at: uvicorn [Consulted on October 2025].
- [8] docker. "docker" [Online], at: docker [Consulted on October 2025].